TermoPlast — Chatbot híbrido: NLP + LLM

Desde V4.3 el chatbot combina tres capas, pensadas para ser **confiables** (nunca se queda sin responder) y **acotadas** (solo habla de TermoPlast):

1. **Clasificador de intención** (TF-IDF + Regresión Logística) — rápido y offline. Rutea la pregunta: datos reales, tema informativo, control o fuera de alcance.
2. **Recuperación semántica (RAG)** — NLP de *information retrieval*: busca en una base de conocimiento (`chatbot/conocimiento.json`) los hechos más parecidos a la pregunta, por similitud coseno (TF-IDF). **No descarga ningún modelo**: funciona offline. Sirve para *fundamentar* (grounding) la respuesta y para detectar temas ajenos (si nada es relevante, no inventa).
3. **Generación con LLM (opcional y pluggable)** — redacta una respuesta en lenguaje natural **a partir del contexto recuperado**, con un *system prompt* que la limita a TermoPlast. Es intercambiable: nube, local (Ollama) o apagado.

Principio clave: el LLM nunca inventa tus datos

Las preguntas que dependen de **datos reales** (estado de un pedido, tu impacto, tu resumen de cuenta, estimaciones) las resuelve el **PHP de forma determinista** contra la base de datos. El LLM solo redacta respuestas **informativas** (materiales, tiempos, cómo pedir, límites, etc.) y siempre **fundamentadas** en el conocimiento. Así nunca "alucina" un estado de pedido o un número.

Backends (en `chatbot/llm_config.json`)

backend	Qué hace	Necesita
`extractivo` *(por defecto)*	Sin modelo ni internet: responde con el mejor fragmento recuperado (RAG).	Nada
`openai`	Genera con una API **compatible con OpenAI** (nube u Ollama local).	base_url + model (+ key si es nube)
`off`	Solo respuestas fijas (como antes).	Nada

El sistema **arranca en `extractivo`**: anda sin configurar nada. La nube es opcional.

Activar la nube (recomendado si hay internet en la defensa)

1. Conseguí una **API key gratis**. Opción simple y rápida: **Groq** (`https://console.groq.com`, nivel gratuito, muy veloz). También sirve OpenAI (con crédito) o Google Gemini (endpoint compatible).

2. Poné la clave en una **variable de entorno** (así no queda en ningún archivo):

- Windows (PowerShell): `setx TERMOPLAST_LLM_KEY "tu_clave_aca"` (reabrí la terminal)

3. Editá `chatbot/llm_config.json`:

```
{
  "backend": "openai",
  "base_url": "https://api.groq.com/openai/v1",
  "model": "llama-3.1-8b-instant"
}
```

   (Para OpenAI: `base_url` `https://api.openai.com/v1`, `model` `gpt-4o-mini`.)

4. Reiniciá el servicio del chatbot (`python app.py`). Listo: ahora redacta con el LLM.

> **Seguridad:** no subas tu API key a GitHub. Usá la variable de entorno `TERMOPLAST_LLM_KEY`; el archivo `llm_config.json` se versiona **sin** clave.

Activar un LLM 100% local (offline, sin key) — Ollama

1. Instalá **Ollama** (`https://ollama.com`) y descargá un modelo chico: `ollama pull llama3.2`
2. En `llm_config.json`:

```
{ "backend": "openai", "base_url": "http://127.0.0.1:11434/v1", "model": "llama3.2", "api_key": "" }
```

3. Reiniciá el chatbot. (En CPU es más lento; conviene una PC con RAM holgada.)

Memoria de conversación

El asistente recuerda los últimos turnos de la charla y se los pasa al LLM, así responde bien a preguntas de seguimiento (“¿y cuánto tardan?”) sin repetir todo. Los **datos reales** (estado de pedidos, impacto) nunca dependen de esa memoria: los resuelve el sistema en el momento.

Verificar tu backend (antes de la defensa)

Corré el autodiagnóstico — no necesita Flask ni la base de datos:

```
python chatbot/probar_llm.py
```

Muestra el backend activo, hace una **prueba de conexión** con el modelo (si es nube u Ollama) y responde algunas preguntas de ejemplo indicando de dónde salió cada respuesta. Si la conexión da ✓, el chatbot va a generar con el LLM.

También: - `GET http://127.0.0.1:5001/salud` muestra el backend activo y si hay clave. - Si el LLM falla o no está, el chat **cae con elegancia** al modo extractivo y sigue respondiendo. La respuesta incluye un campo `fuente` (`extractivo`/`openai`) para saber de dónde salió.

Próximo paso posible

Reemplazar el TF-IDF del RAG por **embeddings** (p. ej. `sentence-transformers` multilingüe) para una comprensión semántica aún mejor. Requiere descargar un modelo (~150 MB) una vez; queda como mejora futura.